DB-Aware Smart Scaffolder (DASS): A Logic-Aware Visual Studio Extension for Automated CRUD UI Generation

Author: Conceptualized by Rashedul Haque Ronjon

Abstract

The DB-Aware Smart Scaffolder (DASS) is a proposed Visual Studio extension that aims to revolutionize database-first (DB-first) ASP.NET Core development by introducing logic-aware, theme-consistent scaffolding. DASS bridges the gap between traditional code generation and intelligent automation by leveraging Roslyn, T4 templates, and EF Core schema introspection. This paper presents the technical architecture, internal logic design, pseudocode, and implementation strategy for DASS, including a modular schema parser and template processor capable of producing Bootstrap-styled, searchable, sortable, and paginated Razor views automatically.

1. Introduction

Traditional scaffolding tools like `dotnet aspnet-codegenerator` and `Scaffold-DbContext` focus primarily on CRUD scaffolding without embedding functional logic such as searching, sorting, or pagination. DASS extends this paradigm by introducing logic awareness and UI consistency at generation time. By integrating Roslyn for syntax manipulation, T4 templates for code layout, and EF Core for data modeling, DASS aims to produce production-ready applications directly from database schemas or Excel-based datasets.

2. System Architecture

DASS is composed of three major layers: 1. **UI & Command Layer** – Visual Studio interface for schema import, template selection, and generation options. 2. **Logic Processor Layer** – Roslyn and T4-powered engine that interprets schema and injects reusable search/sort logic. 3. **Output & Sync Layer** – Manages file creation, regeneration, and schema synchronization using intelligent diffing.

3. Schema Parser Module (Technical Specification)

The Schema Parser is responsible for interpreting database schemas, Excel headers, or JSON data structures and converting them into a unified in-memory model (`EntityDefinition`). This metadata drives code generation across controllers, models, and views.

Pseudocode:

```
function ParseSchema(source): if source.type == 'database': schema =
FetchDatabaseSchema(source.connectionString) elif source.type == 'excel': schema =
ReadExcelHeaders(source.filePath) elif source.type == 'json': schema =
ReadJsonSchema(source.filePath) entities = [] for table in schema.tables: entity =
EntityDefinition() entity.Name = table.name entity.Fields = [] for column in table.columns:
field = FieldDefinition() field.Name = column.name field.Type = MapToCSharpType(column.type)
field.IsPrimaryKey = column.isPrimaryKey field.IsNullable = column.isNullable
entity.Fields.append(field) entities.append(entity) return entities
```

4. Template Processor (T4 + Roslyn Integration)

The Template Processor integrates T4 templates with Roslyn code generation to create controllers, views, and models that inherit reusable logic from shared partials and helper classes. The processor injects pagination, sorting, and search functionality automatically based on the schema metadata.

Pseudocode:

```
function GenerateFiles(entities, templatePath, outputDir): for entity in entities:
controllerTemplate = LoadTemplate(templatePath + "/Controller.tt") viewTemplate =
LoadTemplate(templatePath + "/IndexView.tt") controllerCode =
RenderTemplate(controllerTemplate, entity) viewCode = RenderTemplate(viewTemplate, entity)
```

```
WriteFile(outputDir + "/Controllers/" + entity.Name + "Controller.cs", controllerCode)
WriteFile(outputDir + "/Views/" + entity.Name + "/Index.cshtml", viewCode)
```

5. Roslyn Integration and Syntax Manipulation

Roslyn allows DASS to inject or update logic blocks without breaking custom user code. The `SmartMergeEngine` uses Roslyn's SyntaxWalker API to detect auto-generated sections (delimited by markers such as `// `). It intelligently updates only the generated sections during regeneration, preserving developer modifications.

6. Example Output (Controller Skeleton)

```
[HttpGet] public async Task Index(string search, string sortOrder, int page = 1) { var query
= _context.Orders.AsQueryable(); query = QueryHelpers.ApplySearch(query, search,
nameof(Order.OrderID), nameof(Order.OrderStatus)); query = QueryHelpers.ApplySort(query,
sortOrder); var pagedList = await PaginatedList.CreateAsync(query, page, 10); return
View(pagedList); }
```

7. Conclusion

The DB-Aware Smart Scaffolder represents a step toward intelligent code generation within Visual Studio. It merges low-code convenience with pro-code flexibility, allowing DB-first developers to produce consistent, maintainable, and modern web applications automatically. With proper integration into Visual Studio’s extensibility APIs, DASS could become a cornerstone tool in future .NET ecosystems for enterprise and rapid application development.